

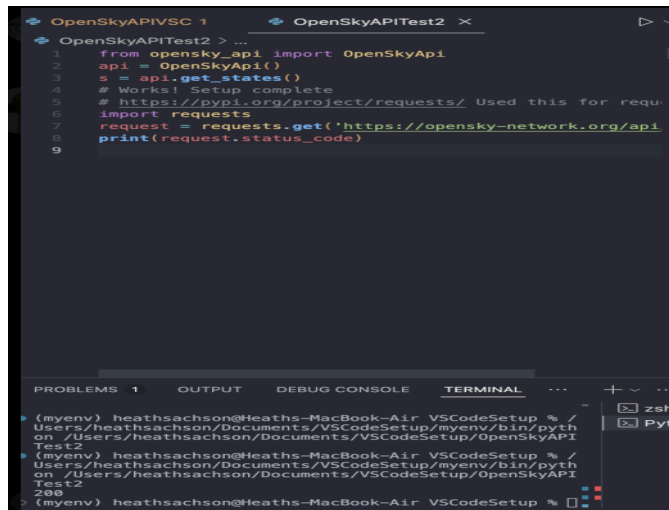
Project 2: Open Sky Flight

Jerry Au

Heath Sachson

Requests Task:

Got status code 200, which is a successful connection.



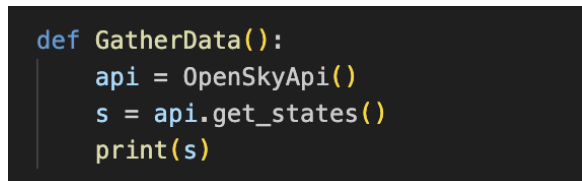
The screenshot shows a VS Code editor with two tabs: 'OpenSkyAPIVSC 1' and 'OpenSkyAPITest2'. The active tab 'OpenSkyAPITest2' contains a Python script with the following code:

```
1 from opensky_api import OpenSkyApi
2 api = OpenSkyApi()
3 s = api.get_states()
4 # Works! Setup complete
5 # https://pypi.org/project/requests/ Used this for requests
6 import requests
7 request = requests.get('https://opensky-network.org/api')
8 print(request.status_code)
9
```

The terminal at the bottom shows the command prompt and the output of the script:

```
(myenv) heathsachson@Heaths-MacBook-Air: VSCodeSetup % python /Users/heathsachson/Documents/VSCoSetup/myenv/bin/python /Users/heathsachson/Documents/VSCoSetup/OpenSkyAPITest2
200
(myenv) heathsachson@Heaths-MacBook-Air: VSCodeSetup %
```

Storing Data:



```
def GatherData():
    api = OpenSkyApi()
    s = api.get_states()
    print(s)
```

This is the method we gathered the data using OpenSkyAPI. We used the UNIX “>” method to push the contents into a csv data file.

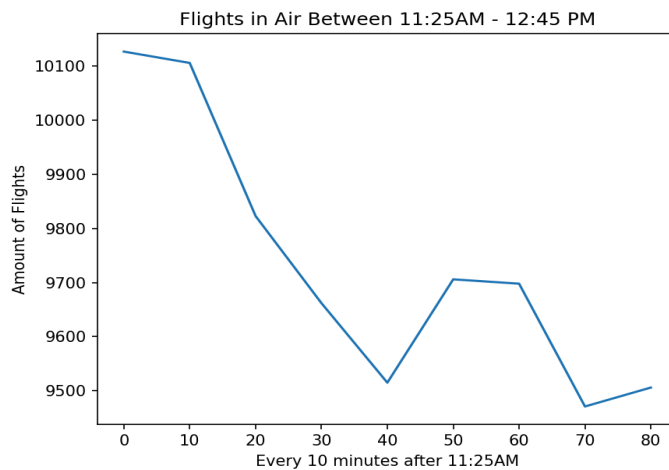
Jerry:

I stored my data and parsed the data using pandas. Specifically when parsing the data, I utilized the pandas function `iloc` to read each row.

Heath:

I stored my data using CSV libraries. I used TextEdit and Table Tool to edit and read my CSVs. I had a little trouble as when I tried to use the Numbers app on my Mac, it automatically converted the .csv to a .numbers file without telling me, so I stopped using it. I stored data for problems 2 and 4 using the above function, but just put it at the top of my file as an import from `opensky_api`.

1. Plot the number of planes in the air over time?



We sampled 9 data sets from 11:25 AM to 12:45 PM taking a sample every 10 minutes. This plot was created using matplotlib

2. How many planes are in the air right now?

I used the first states file we received from the OpenSkyAPI, and made a definition that went through only the 'on_ground' column. I did a simple for row in data block to look through the CSV as a dictionary, and set it to look through rows with the header of 'on_ground'. I ran a count that added 1 to itself whenever a cell with 'False' was read, and returned that count at the end. When a cell reads 'False' in the 'on_ground' column, that means that the plane is currently not on the ground, or rather it is in the air and actively flying. With that, when I returned the count, I got 9686 flights active.

```
count = 0
with open('FlightDataCSVTable.csv', 'r') as f:
    data = csv.DictReader(f)
    # Reads data into dict format

    # Initialize count
    for row in data:
        # Had to make header ' True', as my CSV Reader di
        grounded = row.get('on_ground', '')
        if "False" in grounded:
            count += 1
    return count
```

3. What is the average altitude and speed of active planes?

The average velocity amongst the 9 data sets is 153.88 knots. This was calculated by summing up the velocity of all the flights and taking the average of it.

```
def CalculateAverage(df):
    tempVelocityList = []
    for flights in df.iloc:
        if flights['velocity'] != 'None':
            tempVelocityList.append((float(flights['velocity'])))
    tempVelocityList.pop()
    averageVelocity = sum(tempVelocityList)/len(tempVelocityList)
    return averageVelocity
```

4. Where are the flights concentrated right now?

I used the first states file we received from the OpenSkyAPI, and made a definition that went through both the 'on_ground' column and the 'origin_country' header. I also used the collections module, specifically the Counter function (from <https://docs.python.org/3/library/collections.html>). The Counter function looks through a given dictionary and keeps track of a count of the reoccurring strings. I also used the most_common function in that subclass, which returns the most common element in the Counter count. Normally, if you run something like Counter('banana').most_common(2), it will return 2 2-tuples that are [('a', 2), ('b', 2)], meaning that the 2 most common variables are 'a' and 'b', both appearing twice. So, with that example, I used Counter to keep track of when a country name appeared with the 'False' label in the 'on_ground' column of the CSV file, which meant that the plane was flying. So, I kept track of all that, and returned the 2-tuple of country and count at the end, which was the United States, 6163 flights. This problem was a little easier than problem 2, as it built off of it.

```
for row in data:
    grounded = row.get('on_ground', '')
    country = row.get('origin_country', '')
    if grounded == 'False':
        countrycounter[country] += 1
# Use Counter() docs and commands
#most_common(1) will give the top most frequent item
mostcommon = countrycounter.most_common(1)
if mostcommon:
    country = mostcommon[0]
else:
    print("didn't work")
return country
```